

Auction System — Capstone Project Report

CS2043 · TamTamCatWorks

Team TamTamCatWorks

2026-06-05

Table of contents

1	Giới thiệu & phạm vi	2
2	Thiết kế hệ thống	2
2.1	Các class chính	2
2.2	Design Patterns	3
2.2.1	Factory - <code>ItemCreator</code>	3
2.2.2	Observer - Spring Application Events	3
2.2.3	Singleton - Spring IoC	4
3	Kiến trúc, Module, Tech Stack	4
3.1	Kiến trúc Client-Server & Tech Stack	4
3.2	Áp dụng MVC	4
3.2.1	Server	4
3.2.2	Client	5
4	Các tính năng cơ bản	5
4.1	Quản lý User / Item	5
4.1.1	Quản lý User	5
4.1.2	Quản lý Item	5
4.2	Quản lý Auction	5
4.3	Xử lý ngoại lệ tập trung với <code>@RestControllerAdvice</code>	6
4.4	Xử lý concurrent bidding	6
4.5	Realtime updates	6
5	Tính năng nâng cao	6
5.1	Anti-Sniping	6
5.2	Auto-Bidding	7
5.3	Search & Pagination	7
5.4	Object Storage with MinIO	7
5.5	Admin Panel	7
6	Phân chia công việc	7
6.1	Nguyễn Hoàng Vũ	7

6.2	Nguyễn Mậu Trung Kiên	7
6.3	Phạm Đức Lộc	8
6.4	Nguyễn Khắc Đại	8

1 Giới thiệu & phạm vi

Hệ thống bidding (đấu giá trực tuyến) là một nền tảng phần mềm cho phép nhiều người dùng cùng tham gia cạnh tranh giá để mua một sản phẩm hoặc dịch vụ 1 trong một khoảng thời gian xác định. Dự án này của bọn em triển khai theo kiến trúc Client-Server. Với Client là Spring RestClient, Server là Springboot application. Dự án có được các tính năng chính sau đây:

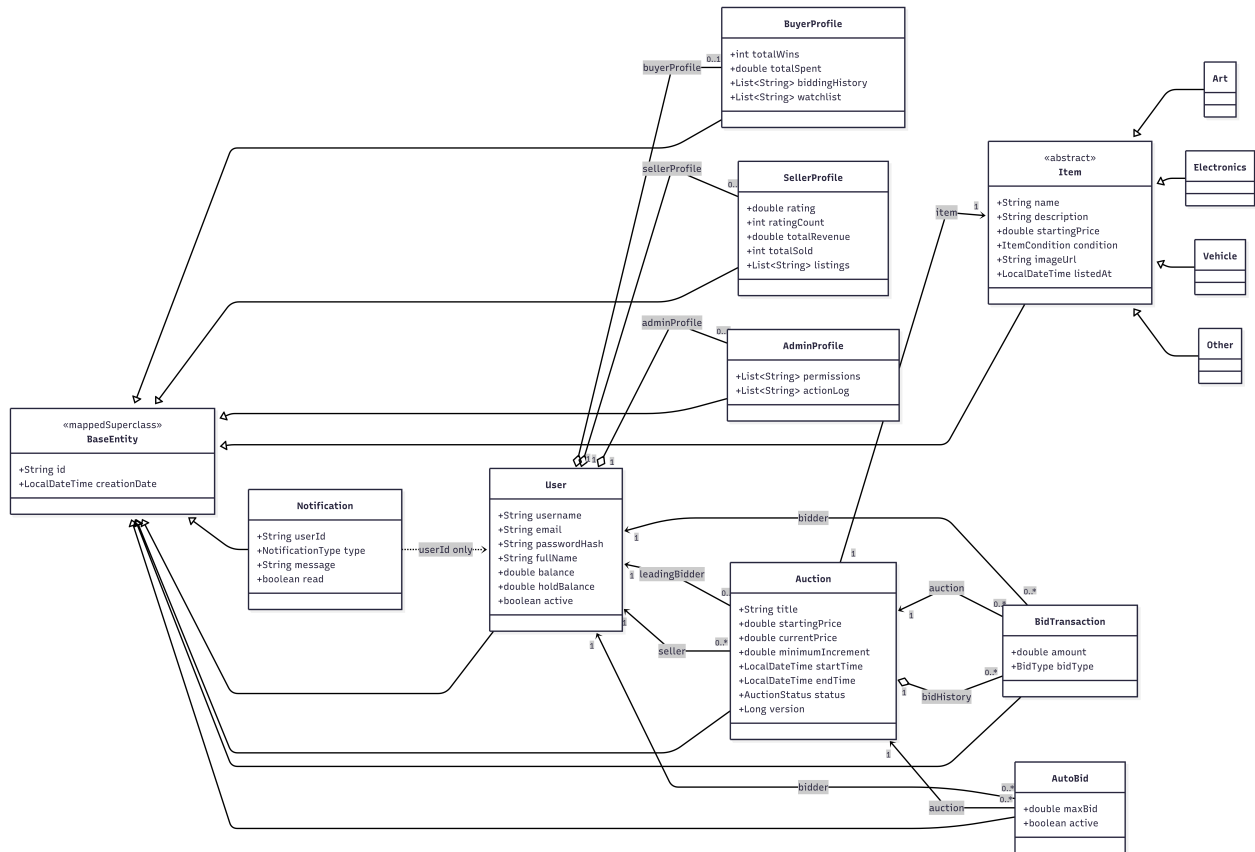
- Quản lý người dùng với các profile: BuyerProfile, SellerProfile, AdminProfile
- Quản lý đấu giá, tự động mở đấu giá với SpringScheduler
- RealTime update qua WebSockets (STOMP over SockJS) - SpringBoot Websockets
- Đấu giá concurrency-safe nhờ optimistic locking của jpa/hibernate, synchronized trên các method của Auction, @Transactional của spring, tạm giữ số dư qua hold funds
- Các chức năng nâng cao: anti-sniping, auto-bidding, search & filter với pagination control, object storage với MinIO
- JavaFX desktop client

Hệ thống gồm 6 module Maven: `shared`, `model`, `persist`, `service`, `api`, và `client`.

2 Thiết kế hệ thống

2.1 Các class chính

Tất cả các domain model đều được định hình dựa trên một abstract superclass có tên là `BaseEntity`. Biểu đồ lớp dưới đây miêu tả các mối quan hệ chính:



Các điểm đáng lưu ý: - BaseEntity được gắn annotation `@MappedSuperclass` và `@GeneratedValue(strategy = GenerationType.UUID)` để tự động tạo ID, giúp đảm bảo các định danh là duy nhất trên toàn hệ thống mà không cần phụ thuộc vào database. - Annotation `@PrePersist` sẽ tự động persist `CreationDate` vào thời điểm tạo. - Auction có member `@Version` (Long version) nhằm phục vụ optimistic locking của JPA.

2.2 Design Patterns

2.2.1 Factory - ItemCreator

Itemcreator áp dụng mẫu thiết kế Factory Method. Các lớp khởi tạo cụ thể (ArtCreator, ElectronicsCreator, VehicleCreator, OtherCreator) là các SpringBean được annotate `@Component`. Chúng sẽ được tự động load vào ItemService khi ứng dụng khởi tạo.

2.2.2 Observer - Spring Application Events

Hệ thống sử dụng ApplicationEventPublisher của Spring. Các Domain events được viết dưới dạng các Java Record (immutable).

Các publisher gồm BidService và AuctionService. Nhiều listener khác nhau sẽ phản hồi độc lập:

Listener	Annotation	Responsibility
NotificationService	@EventListener	Persists notification records to DB
NotificationBroadcastListener	@EventListener	Pushes real-time WebSocket messages
AutoBidService	@TransactionalEventListener	Triggers competing auto-bids
UserStateBroadcastListener	@EventListener	Pushes updated user balance/state

2.2.3 Singleton - Spring IoC

Tất cả các Bean được đánh dấu @Service, @Component, và @Configuration đều là các singleton do Spring quản lý. Container IoC của Spring sẽ chịu trách nhiệm quản lý vòng đời, dependency injection, và thread-safety guarantees thông qua proxying mechanism của Spring.

3 Kiến trúc, Module, Tech Stack

3.1 Kiến trúc Client-Server & Tech Stack

Component	Technology	Role
Server (API)	Spring Boot 3.2.5, Java 21	REST API + WebSocket broker
Client	JavaFX 21 + AtlantaFX	Desktop GUI
Database	PostgreSQL 16	Persistent storage
Object Storage	MinIO (S3-compatible)	Auction item images
Containerization	Docker Compose	Dev environment orchestration

3.2 Áp dụng MVC

3.2.1 Server

Server phân thành 5 module khác nhau như sau:

Shared Module (DTO Request/Response - Java Record)

API Module	Service Module	Persist Module	Model Module
Controllers	Business	Kho lưu trữ	JPA Entities
WebSocket	Mappers	(Spring Data)	
Xử lý lỗi	Events		
Bảo mật	Lưu trữ ảnh		

Request flow:

1. Client gửi yêu cầu HTTP đến **@RestController**
2. Controller trích xuất thông tin phiên người dùng, sau đó ủy quyền xử lý cho **@Service**(ví dụ: **AuctionService**)
3. Tầng Service xử lý business logic, đồng thời sử dụng **@Repository** để persist vào database
4. Dữ liệu phản hồi được ánh xạ ngược lại thông qua **MapStruct** & trả về cho client dưới dạng chuỗi JSON

3.2.2 Client

Client sử dụng:

- **View:** FXML dưới `client/src/main/resources/fxml/`
- **Controller:** inject FXML
- **Model:** các dto trong `shared`

4 Các tính năng cơ bản

4.1 Quản lý User / Item

4.1.1 Quản lý User

- Đăng ký, đăng nhập: Xác thực qua BCrypt, Spring Security quản lý sessions

4.1.2 Quản lý Item

Item được tạo bởi Factory đã được mô tả ở phần trước. Lớp **ItemService** là entry point cho toàn bộ quy trình:

1. Client gửi một yêu cầu **ItemRequest** đi kèm với loại sản phẩm **itemType** cụ thể (ví dụ: “ART”, “ELECTRONICS”).
2. **ItemService** chọn factory từ registry của **ItemCreator**.
3. **ItemCreator** sẽ validate các miền trước khi tạo Item
4. Item sau đó sẽ được lưu qua **ItemRepository**.

4.2 Quản lý Auction

- **AuctionScheduler** là **SpringScheduler**, tự động kiểm tra các cuộc đấu giá quá hạn/đến lúc mở & tự động đóng/mở chúng.
- Khi đóng cuộc đấu giá, **AuctionService.close()** sẽ chuyển auction thành closed, consume holdbalance của leading bidder, cộng số tiền ấy vào tài khoản seller, và phát các event liên quan

4.3 Xử lý ngoại lệ tập trung với @RestControllerAdvice

Lớp `GlobalExceptionHandler` cho phép code ở tầng dưới chỉ cần ném ra các ngoại lệ tiêu chuẩn của Java (chẳng hạn như `IllegalArgumentException`, `NoSuchElementException`, `IllegalStateException`), và Spring sẽ tự động chuyển chúng sang HTTP status code với json giống nhau.

4.4 Xử lý concurrent bidding

1. JPA sẽ phát hiện version mismatch ở lượt commit thứ hai và ném `OptimisticLockException`. `@Transactional` của Spring sẽ tự động rollback giao dịch của bên thất bại.
2. Các domain model bổ sung từ khóa `synchronized` vào tất cả các phương thức làm thay đổi trạng thái dữ liệu (`open()`, `close()`, `cancel()`, `recordBid()`, `extendEndTime()`)
3. `@Transactional` trong các giao dịch do Spring quản lý. Phương thức `BidService.placeBid()` đóng gói toàn bộ luồng xử lý đấu giá.

4.5 Realtime updates

Khi có `SpringEvents` nhất định, ví dụ như `BidEvent`. Các `Websocket Listener` sẽ publish những event ấy qua websockets. Client sẽ subscribes qua các topic/queue nhất định khi connect với server, và sẽ disconnect ws khi user đăng xuất.

Một số các topic:

Topic	Payload	Scope
/topic/auctions/{id}	AuctionPriceUpdate	Broadcast to all viewers
/topic/auctions/{id}/status	AuctionStatusUpdate	Broadcast on status change
/user/queue/notifications	NotificationResponse	Private to specific user
/topic/user-state/{userId}	UserResponse	User balance refresh

5 Tính năng nâng cao

5.1 Anti-Sniping

Xử lý ở trong `BidService.placeBid()`:

```
LocalDateTime snipeWindowStart = auction.getEndTime()
    .minusSeconds(antiSnipe.windowSeconds());
if (!LocalDateTime.now().isBefore(snipeWindowStart)) {
    auction.extendEndTime(antiSnipe.extensionSeconds());
}
```

5.2 Auto-Bidding

Tất cả các bid dù là Manual hay Auto đều publish một `BidEvent`. `onBidPlaced` ở `AutoBidService` có nhiệm vụ xử lý chúng.

Thuật toán: - Load tất cả các auto-bids cho Auction, sort bởi `maxBid DESC` với tiebreaker là `creationDate ASC` - pop phần tử đầu của danh sách vào `best` - nếu `best` là leader, deactivate tất cả các auto-bids còn lại với mức giá bé hơn mức giá tiếp theo `minimumIncrement`. Nếu queue empty thì return. Nếu không thì leader phải counter bid với người đặt giá thứ 2.

5.3 Search & Pagination

- `AuctionRepository` implements **JPQL joins** và **Pageable** của Spring Data. Server sẽ paginate nhờ **Pageable** của `SpringData` và sẽ serialize qua DTO `PageResponse<T>`
- API expose endpoint ở `GET /auctions/search?q=&status=&category=&page=0&size=10`
- Các cơ chế filter được implement ở client.

5.4 Object Storage with MinIO

Các ảnh khi upload được lưu trong MinIO, storage-server tương thích với S3 của Amazon. `ImageStorageService` có tính năng:

- tự động khởi tạo bucket và set public-read S3 policy nếu chưa có
- upload ảnh với tên là UUID được tự động tạo
- trả lại URL của ảnh đó cho client.

5.5 Admin Panel

- Có Panel để admin quản lý auction. User có thể bị suspend. Suspend user bị filter qua `SecurityChain` của Spring & không tương tác được với hệ thống. Khi suspend, toàn bộ các active auction của User bị cancel qua `AuctionService.cancel()`

6 Phân chia công việc

6.1 Nguyễn Hoàng Vũ

- Quản lý dự án, config môi trường & lên ý tưởng các tính năng. Review code & Sửa lỗi. Làm Client & Thiết kế giao diện. Object Storage, Search & Paginaton.

6.2 Nguyễn Mậu Trung Kiên

- Model, persist, service cho `User`, `Service`, `Auto-Bid`, `Anti-Sniping`, `AuctionScheduler`, `Websockets` cho `Service`

6.3 Phạm Đức Lộc

- Model, persist, service cho `Item`, làm frontend & backend cho `Admin`

6.4 Nguyễn Khắc Đại

- Model, persist, service cho `Auction`, `BidTransaction`. Auto-bid.